МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ ВИЩИЙ НАВЧАЛЬНИЙ ЗАКЛАД
«УЖГОРОДСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ»
ІНЖЕНЕРНО-ТЕХНІЧНИЙ ФАКУЛЬТЕТ
КАФЕДРА КОМП'ЮТЕРНИХ СИСТЕМ ТА МЕРЕЖ

Звіт
до лабораторної роботи №5
з дисципліни «PSoC»

Студент 4-го курсу
спеціальності F7 –
«Комп'ютерна інженерія»
Глеба Василь Васильович

м. Ужгород – 2026 р

ЛАБОРАТОРНА РОБОТА №5

Використання зсувних регістрів та семисегментних світлодіодних індикаторів

у інтегрованому середовищі PSoC® Creator™

Мета: Ознайомитися з принципами роботи регістрів зсуву та семисегментних світлодіодних індикаторів у мікроконтролерних системах на базі PSoC. Набути практичних навичок створення проєктів у інтегрованому середовищі PSoC® Creator™, налаштування апаратних компонентів для керування індикаторами через зсувні регістри та реалізації ефективного виведення даних.

Самостійні завдання:

− Модифікувати код таким чином, аби на семисегментний дисплей виводився номер натиснутої кнопки матричної клавіатури.

− Модифікувати код, додавши можливість відображення символів: A, b, C, d, E, F.

Виконання:

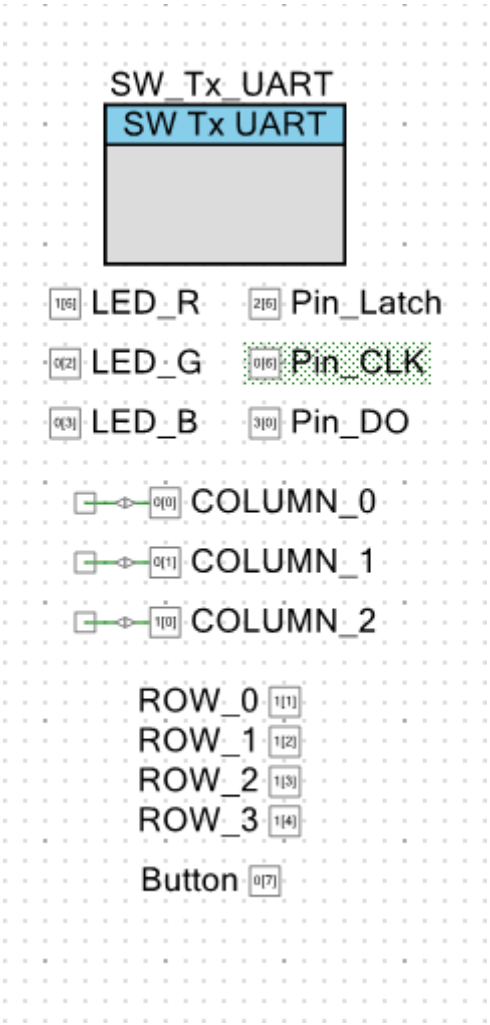

Рисунок 1 -Скриншот редактора схеми проєкту.

Name	Port	Pin	Lock
\SW_Tx_UART:tx\	P2[7]	9	☑
Button	P0[7]	31	☑
COLUMN_0	P0[0]	24	☑
COLUMN_1	P0[1]	25	☑
COLUMN_2	P1[0]	37	☑
LED_B	P0[3]	27	☑
LED_G	P0[2]	26	☑
LED_R	P1[6]	43	☑
Pin_CLK	P0[6]	30	☑
Pin_DO	P3[0]	11	☑
Pin_Latch	P2[6]	8	☑
ROW_0	P1[1]	38	☑
ROW_1	P1[2]	39	☑
ROW_2	P1[3]	40	☑
ROW_3	P1[4]	41	☑

Рисунок 2 - Скриншот редактора ресурсів.

Висновок.Ознайомитися з принципами роботи регістрів зсуву та семисегментних світлодіодних індикаторів у мікроконтролерних системах на базі PSoC. Набути практичних навичок створення проєктів у інтегрованому середовищі PSoC® Creator™, налаштування апаратних компонентів для керування індикаторами через зсувні регістри та реалізації ефективного виведення даних.

Лістинг коду:

```
#include "project.h"
static uint8_t LED_NUM[] = {
    0xC0,   //0
    0xF9,   //1
    0xA4,   //2
    0xB0,   //3
    0x99,   //4
    0x92,   //5
    0x82,   //6
    0xF8,   //7
    0x80,   //8
    0x90,   //9
    0xBF,   //-
    0x88,   //A
    0x83,   //b
    0xC6,   //C
    0xA1,   //d
    0x86,   //E
```

```
    0x8E,   //F     0x7F    //.
};
/* send data function */
static void FourDigit74HC595_sendData(uint8_t data) {
    for(uint8_t i = 0; i < 8; i++) {
        if(data & (0x80 >> i))
        {
            Pin_DO_Write(1);
        }
        else
        {
            Pin_DO_Write(0);
        }
        Pin_CLK_Write(1);
        Pin_CLK_Write(0);
    }
}

static void FourDigit74HC595_sendOneDigit(uint8_t position, uint8_t digit,
uint8_t dot)
{
    if(position >= 8)   // out of range, clear 7-segment display
    {
        FourDigit74HC595_sendData(0xFF);
        FourDigit74HC595_sendData(0xFF);
    }
    FourDigit74HC595_sendData(0xFF & ~(1 << position));
    if(dot) // if dot is needed
    {
        FourDigit74HC595_sendData(LED_NUM[digit] & 0x7F);
    }     else
    {
        FourDigit74HC595_sendData(LED_NUM[digit]);
    }
    Pin_Latch_Write(1); // Latch shift register
    Pin_Latch_Write(0);
}

/* function of drive mode configuration */
static void (*COLUMN_x_SetDriveMode[3])(uint8_t mode) = {
    COLUMN_0_SetDriveMode,
    COLUMN_1_SetDriveMode,
    COLUMN_2_SetDriveMode
};
/* column write function */
static void (*COLUMN_x_Write[3])(uint8_t value) = {
    COLUMN_0_Write,
    COLUMN_1_Write,
    COLUMN_2_Write
};
/* read row function */
static uint8 (*ROW_x_Read[4])() = {
    ROW_0_Read,
    ROW_1_Read,
    ROW_2_Read,
    ROW_3_Read
};

/* [ROW][COLUMN] */
static uint8_t keys[4][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9},
    {10, 0, 11},
```

```
};

/* Password settings */
#define PASSWORD_LENGTH 3
static const uint8_t PASSWORD[PASSWORD_LENGTH] = {1, 2, 3}; // Встановіть
свій пароль тут
static uint8_t password_buffer[PASSWORD_LENGTH];
static uint8_t password_index = 0;
static uint8_t password_locked = 1; // 1 = заблоковано, 0 = розблоковано

/* matrix initialization function */
static void initMatrix()
{
    for(int column_index = 0; column_index < 3; column_index++)
    {
         COLUMN_x_SetDriveMode[column_index](COLUMN_0_DM_DIG_HIZ);
    }
}

/* keys matrix read function */
static void readMatrix()
{
    /* define the length of a row and column */
    uint8_t row_counter = sizeof(ROW_x_Read)/sizeof(ROW_x_Read[0]);
    uint8_t column_counter =
sizeof(COLUMN_x_Write)/sizeof(COLUMN_x_Write[0]);
    /* column: iterate the columns */
    for(int column_index = 0; column_index < column_counter; column_index++)
{
        COLUMN_x_SetDriveMode[column_index](COLUMN_0_DM_STRONG);
        COLUMN_x_Write[column_index](0);
        /* row: interate throught the rows */
        for(int row_index = 0; row_index < row_counter; row_index++)
        {
            keys[row_index][column_index] = ROW_x_Read[row_index]();
        }
        /* disable the column */
        COLUMN_x_SetDriveMode[column_index](COLUMN_0_DM_DIG_HIZ);
    }
}

/* Function to check password */
static void checkPassword()
{
    uint8_t correct = 1;
    for(int i = 0; i < PASSWORD_LENGTH; i++)
    {
        if(password_buffer[i] != PASSWORD[i])
        {
            correct = 0;
            break;
        }
    }

    if(correct)
    {
        SW_Tx_UART_PutString("Access allowed");
        SW_Tx_UART_PutCRLF();
        password_locked = 0;
        // Білий світлодіод при розблокуванні
        LED_R_Write(0);
        LED_G_Write(0);
        LED_B_Write(0);
        CyDelay(1000);
```

```
    }
    else
    {
        SW_Tx_UART_PutString("Access denied");
        SW_Tx_UART_PutCRLF();
        password_locked = 1;
    }

    password_index = 0;
}

/* Function to set LED color based on button */
static void setLEDColor(uint8_t button_value)
{
    switch(button_value)
    {
        case 1:  // Red
        case 7:
        FourDigit74HC595_sendOneDigit(0, button_value, 0);
            LED_R_Write(0);
            LED_G_Write(1);
            LED_B_Write(1);
            break;

        case 2:  // Green

        case 8:
        FourDigit74HC595_sendOneDigit(0, button_value, 0);
            LED_R_Write(1);
            LED_G_Write(0);
            LED_B_Write(1);
            break;

        case          3:          //          Blue
        FourDigit74HC595_sendOneDigit(0, 3, 0);
        case 9:
        FourDigit74HC595_sendOneDigit(0, button_value, 0);
            LED_R_Write(1);
            LED_G_Write(1);
            LED_B_Write(0);
            break;

        case 4:  // Yellow (Red + Green)
        case 10: // *
        FourDigit74HC595_sendOneDigit(0, button_value, 0);
            LED_R_Write(0);
            LED_G_Write(0);
            LED_B_Write(1);
            break;

        case 5:  // Purple (Red + Blue)
        case 0:
        FourDigit74HC595_sendOneDigit(0, button_value, 0);
            LED_R_Write(0);
            LED_G_Write(1);
            LED_B_Write(0);
            break;

        case 6:  // Cyan (Green + Blue)
        case 11: // #
        FourDigit74HC595_sendOneDigit(0, button_value, 0);
            LED_R_Write(1);
            LED_G_Write(0);
            LED_B_Write(0);
```

```
            break;

        default:
        FourDigit74HC595_sendOneDigit(8, 0, 0);
            // Turn off LED (Black)
            LED_R_Write(1);
            LED_G_Write(1);
            LED_B_Write(1);
            break;
    }
}

/* Function to get button name */
static void printButtonName(uint8_t button_value)
{
    switch(button_value)
    {
        case 0:
            SW_Tx_UART_PutString("Button 0 pressed");
            break;
        case 1:
            SW_Tx_UART_PutString("Button 1 pressed");
            break;
        case 2:
            SW_Tx_UART_PutString("Button 2 pressed");
            break;
        case 3:
            SW_Tx_UART_PutString("Button 3 pressed");
            break;
        case 4:
            SW_Tx_UART_PutString("Button 4 pressed");
            break;
        case 5:
            SW_Tx_UART_PutString("Button 5 pressed");
            break;
        case 6:
            SW_Tx_UART_PutString("Button 6 pressed");
            break;
        case 7:
            SW_Tx_UART_PutString("Button 7 pressed");
            break;
        case 8:
            SW_Tx_UART_PutString("Button 8 pressed");
            break;
        case 9:
            SW_Tx_UART_PutString("Button 9 pressed");
            break;
        case 10:
            SW_Tx_UART_PutString("Button * pressed");
            break;
        case 11:
            SW_Tx_UART_PutString("Button # pressed");
            break;
        default:
            break;
    }
    SW_Tx_UART_PutCRLF();
}

int main(void)
{
    CyGlobalIntEnable; /* Enable global interrupts. */
    FourDigit74HC595_sendOneDigit(0, 0x88, 1);
    SW_Tx_UART_Start();
```

```
SW_Tx_UART_PutCRLF();
SW_Tx_UART_PutString("Software Transmit UART");
SW_Tx_UART_PutCRLF();
SW_Tx_UART_PutString("Enter password to unlock");
SW_Tx_UART_PutCRLF();

initMatrix();

uint8_t last_state = 255; // Ніяка кнопка не натиснута
uint8_t button_pressed = 0;

// Початковий стан - білий світлодіод (система заблокована)
LED_R_Write(0);
LED_G_Write(0);
LED_B_Write(0);

for(;;)
{
    readMatrix();
    button_pressed = 0;

    // Сканування всіх кнопок
    for(int row = 0; row < 4; row++)
    {
        for(int col = 0; col < 3; col++)
        {
            if(keys[row][col] == 0) // Кнопка натиснута
            {
                uint8_t button_value = (row == 3 && col == 1) ? 0 :
                                       (row == 3 && col == 0) ? 10 :
                                       (row == 3 && col == 2) ? 11 :
                                       (row * 3 + col + 1);

                if(last_state != button_value)
                {
                    last_state = button_value;
                    printButtonName(button_value);

                    // Якщо система заблокована, збираємо пароль
                    if(password_locked)
                    {
                        password_buffer[password_index] = button_value;
                        password_index++;
                        if(password_index >= PASSWORD_LENGTH)
                        {
                            checkPassword();
                        }
                    }
                    else
                    {
                        // Система розблокована, змінюємо колір
                        setLEDColor(button_value);

                    }
                }
                button_pressed = 1;
                break;
            }
        }
        if(button_pressed) break;
    }

    // Якщо жодна кнопка не натиснута
    if(!button_pressed)
```

```
        {
            if(last_state != 255)
            {
                last_state = 255;

                if(password_locked)
                {
                    // Білий світлодіод, коли система заблокована і кнопка
відпущена
                    LED_R_Write(0);
                    LED_G_Write(0);
                    LED_B_Write(0);
                }
                else
                {
                    FourDigit74HC595_sendOneDigit(8, 0, 0);
                    // Вимкнути світлодіод (Black), коли система розблокована
і кнопка відпущена
                    LED_R_Write(1);
                    LED_G_Write(1);
                    LED_B_Write(1);
                }
            }
        }

        CyDelay(50); // Debounce delay
    }
}
```